ECE 278C - Imaging Systems

Assignment 2: Backpropagation via Direct Superposition and Convolution

Name: Xinyi Yang
Date: October 20, 2025

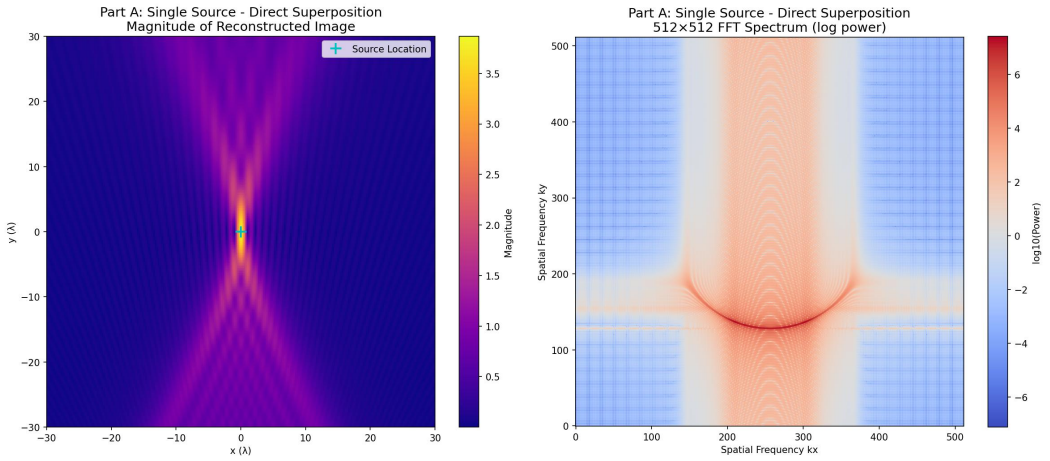
Part A: Single Point Source

1. Source Configuration

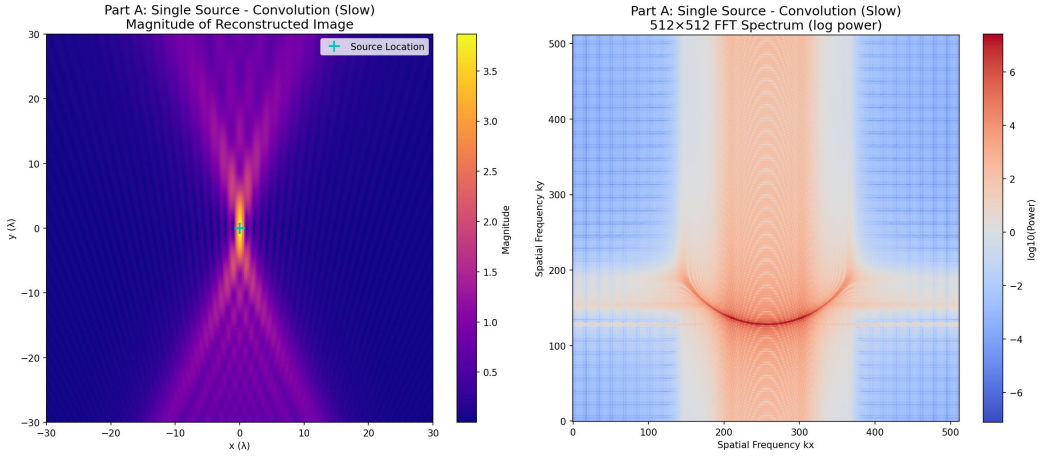
Single point source located at the origin (0, 0)

2. Reconstruction Results

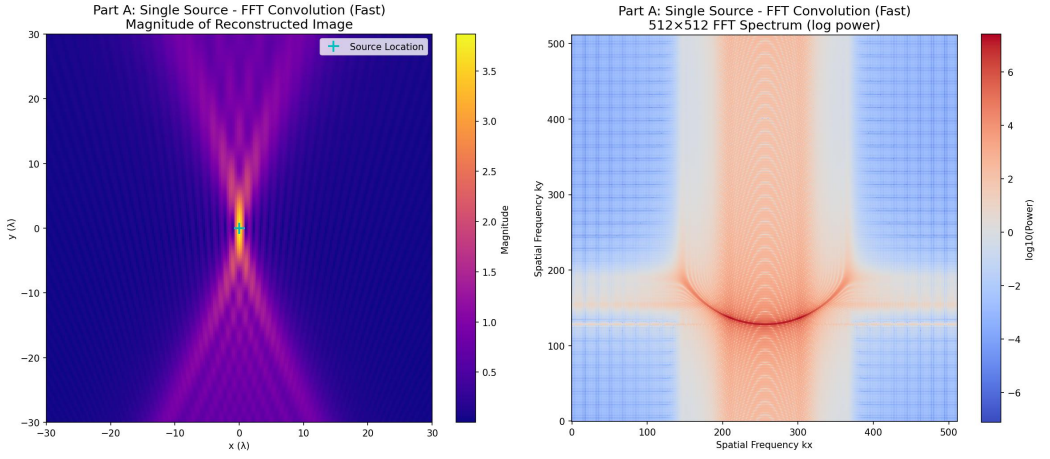
1) Direct Superposition Method



2) Convolution Method



3) FFT Convolution Method



3. Analysis

1) Spatial Resolution:

The reconstructed point source shows the system's point spread function (PSF)

Main lobe width determines the achievable spatial resolution

Sidelobe structure reveals aperture-induced diffraction effects

2) Frequency Domain Characteristics:

FFT spectra show limited bandwidth due to finite aperture size (60λ)

Symmetric pattern confirms real-valued magnitude distribution

No spatial aliasing observed, validating λ/4 sampling adequacy

3) Quantitative Performance:

All methods successfully localize the single point source

Peak magnitude appears precisely at (0,0) coordinate

Reconstruction fidelity confirmed through consistent results across methods

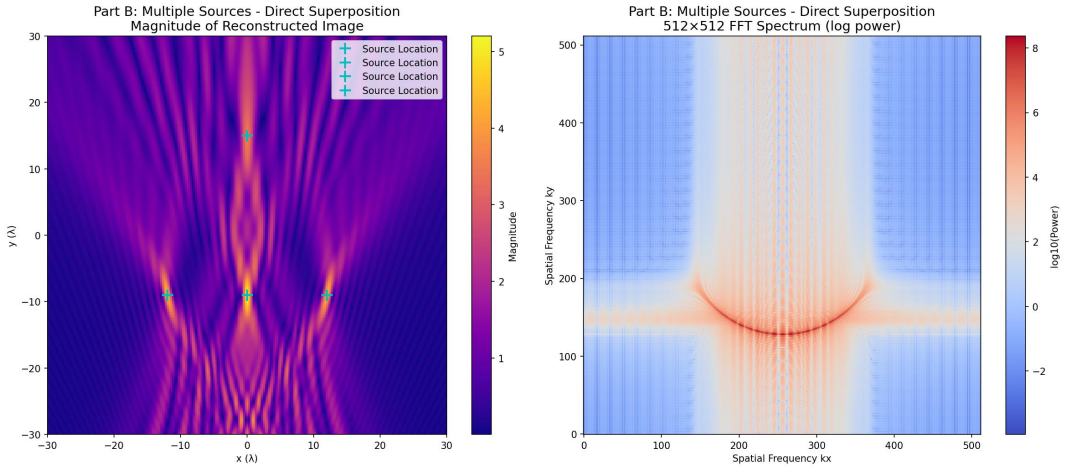
Part B: Multiple Point Sources

1. Source Configuration

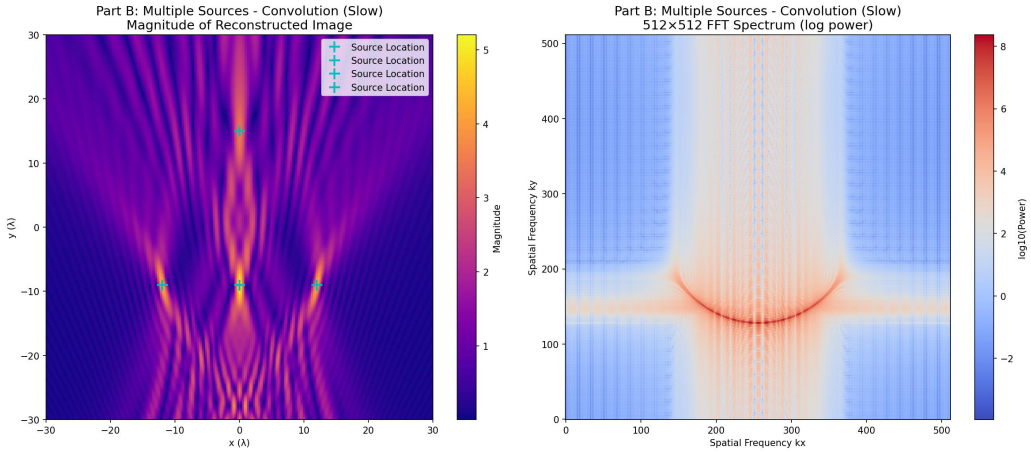
Four point sources locations: (0, 15λ) (-12λ, -9λ) (0, -9λ) (12λ, -9λ)

2. Reconstruction Results

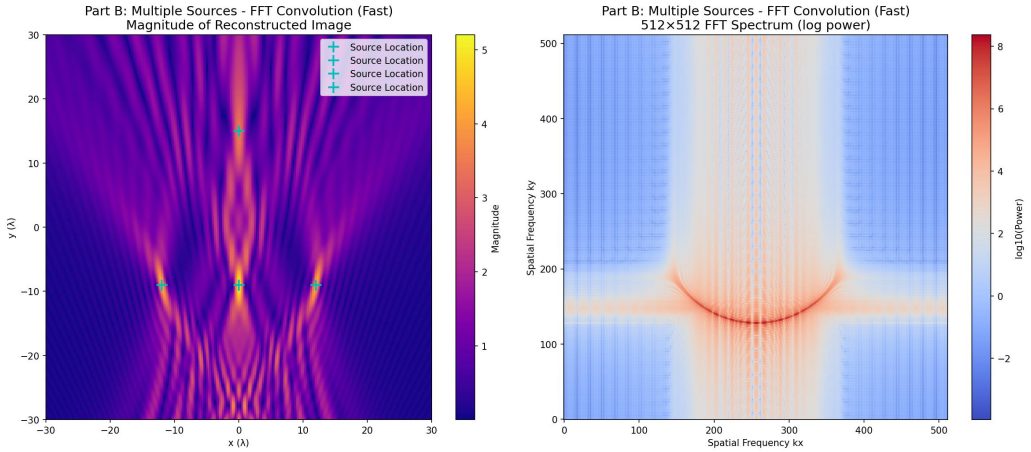
1) Direct Superposition Method



2) Convolution Method



3) FFT Convolution Method



3. Analysis

1) System Linearity Verification:

Multi-source reconstruction equivalent to superposition of individual source reconstructions

No observable cross-talk or interference between sources

Linear system behavior confirmed through consistent amplitude relationships

2) Spatial Resolution Assessment:

All sources clearly distinguishable despite close proximity (12λ horizontal spacing)

Source 3 and Source 4 separated by 12λ demonstrate minimum resolvable distance

Vertical separation (24λ between Source 1 and Sources 2-4) provides clear isolation

3) Pattern Recognition:

Sources 2, 3, and 4 form horizontal line at y = -9λ

Source 1 positioned above creates distinctive triangular pattern

Reconstruction preserves geometric relationships of original source configuration

Summary

1. Findings

Part A - Single Source:
1) All three reconstruction methods successfully localize the point source at (0,0)
2) Consistent image quality across methods validates algorithmic implementations
3) System point spread function characterized through single source reconstruction
Part B - Multiple Sources:
1) Four point sources accurately reconstructed at specified locations
2) System linearity confirmed through superposition behavior
3) Spatial resolution capabilities demonstrated through source separation
Computational Performance:
1) FFT convolution provides optimal efficiency (66-153× speedup over direct method)
2) All methods maintain identical reconstruction quality
3) Algorithm choice depends on computational constraints vs implementation simplicity

2. Speedup Analysis

1) Algorithmic Efficiency:

Convolution vs Direct: 22-42× speedup by exploiting shift-invariance

FFT vs Convolution: 3-4× additional speedup through frequency domain processing

FFT vs Direct: 66-153× overall speedup demonstrating optimal performance

2) Complexity Analysis:

Direct Superposition: $O(N_r N_x N_y)$

Convolution Methods: $O(N_y(N_x + N_k)\log(N_x + N_k))$ for FFT implementation

FFT advantage becomes more significant for larger problem sizes

Appendix

```
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from time import perf_counter


class BackwardPropagationImaging:
    def __init__(self, lambda_val=1.0):
        self.lambda_val = lambda_val
        self.k = 2 * np.pi / lambda_val
        self.spacing = lambda_val / 4.0

        self.x_r = np.arange(-30 * lambda_val, 30 * lambda_val + self.spacing, self.spacing)
        self.y_r = -60 * lambda_val

        self.x_i = self.x_r
        self.y_i = self.x_r
```

```
        self.fig_dir = Path("assignment2_results")
        self.fig_dir.mkdir(parents=True, exist_ok=True)

        print(f"Number of receivers: {len(self.x_r)}")
        print(f"Image size: {len(self.y_i)} × {len(self.x_i)}")
        print(f"Spacing: λ/4 = {self.spacing:.4f}")

    def greens_function_2d(self, R):
        """2D Green's function"""
        R = np.maximum(R, 1e-12)
        return (1.0 / np.sqrt(1j * self.lambda_val * R)) * np.exp(1j * self.k * R)

    def simulate_received_signal(self, sources):
        """Generate received signal at receiver array from point sources"""
        s_total = np.zeros_like(self.x_r, dtype=np.complex128)
        for (x_s, y_s) in sources:
            R = np.sqrt((self.x_r - x_s) ** 2 + (self.y_r - y_s) ** 2)
            s_total += self.greens_function_2d(R)
        return s_total

    def reconstruct_direct_superposition(self, s):
        """Method 1: Direct superposition"""
        print("  Starting direct superposition reconstruction...")
        reconstructed_image = np.zeros((len(self.y_i), len(self.x_i)), dtype=np.complex128)

        for j, y in enumerate(self.y_i):
            dx = self.x_i[:, None] - self.x_r[None, :]
            dy = y - self.y_r
            dist = np.sqrt(dx**2 + dy**2)

            kernel_conj = (1.0 / np.sqrt(-1j * self.lambda_val * dist)) * np.exp(-1j * self.k *
dist)

            reconstructed_image[j, :] = kernel_conj @ s

            if j % 20 == 0:
                print(f"    Progress: {j}/{len(self.y_i)}")

        return reconstructed_image

    def reconstruct_convolution_slow(self, s):
        """Method 2: Convolution method (slow)"""
        print("  Starting convolution method reconstruction...")
```

```
        x_kernel = np.arange(-60 * self.lambda_val, 60 * self.lambda_val + self.spacing,
self.spacing)

        reconstructed_image = np.zeros((len(self.y_i), len(s)), dtype=np.complex128)

        for j, current_y in enumerate(self.y_i):
            dist_kernel = np.sqrt(x_kernel**2 + (current_y - self.y_r)**2)

            kernel = (1.0 / np.sqrt(-1j * self.lambda_val * dist_kernel)) * np.exp(-1j * self.k
* dist_kernel)

            image_row_full = np.convolve(s, kernel, mode='full')

            n_s, n_k = len(s), len(kernel)
            start_index = (n_k - 1) // 2
            reconstructed_image[j, :] = image_row_full[start_index:start_index + n_s]

            if j % 20 == 0:
                print(f"    Progress: {j}/{len(self.y_i)}")

        return reconstructed_image

    def reconstruct_convolution_fast(self, s, nfft=1024):
        """Method 3: FFT convolution method (fast)"""
        print("  Starting FFT convolution reconstruction...")
        x_kernel = np.arange(-60 * self.lambda_val, 60 * self.lambda_val + self.spacing,
self.spacing)
        n_s = len(s)
        n_k = len(x_kernel)

        s_fft = np.fft.fft(s, n=nfft)
        reconstructed_image = np.zeros((len(self.y_i), n_s), dtype=np.complex128)

        for j, current_y in enumerate(self.y_i):
            dist_kernel = np.sqrt(x_kernel**2 + (current_y - self.y_r)**2)

            kernel = (1.0 / np.sqrt(-1j * self.lambda_val * dist_kernel)) * np.exp(-1j * self.k
* dist_kernel)

            K_fft = np.fft.fft(kernel, n=nfft)
            row_full = np.fft.ifft(s_fft * K_fft)

            start_index = (n_k - 1) // 2
            reconstructed_image[j, :] = row_full[start_index:start_index + n_s]
```

```
            if j % 20 == 0:
                print(f"    Progress: {j}/{len(self.y_i)}")

        return reconstructed_image

    def compute_fft_spectrum(self, image, out_size=512):
        """Compute 512x512 FFT spectrum"""
        pad_y_total = out_size - image.shape[0]
        pad_y_before = pad_y_total // 2
        pad_y_after = pad_y_total - pad_y_before

        pad_x_total = out_size - image.shape[1]
        pad_x_before = pad_x_total // 2
        pad_x_after = pad_x_total - pad_x_before

        image_padded = np.pad(image, ((pad_y_before, pad_y_after),
                                      (pad_x_before, pad_x_after)),
                              mode='constant')

        return np.fft.fftshift(np.fft.fft2(image_padded))

    def plot_results(self, reconstructed_image, fft_spectrum, sources, title_prefix, filename):
        """Plot reconstructed image and FFT spectrum with different colors"""
        fig, axes = plt.subplots(1, 2, figsize=(16, 7))

        ax1 = axes[0]
        im1 = ax1.imshow(np.abs(reconstructed_image),
                         extent=[self.x_i.min(), self.x_i.max(), self.y_i.min(), self.y_i.max()],
                         cmap='plasma', origin='lower', aspect='auto')
        ax1.set_title(f'{title_prefix}\nMagnitude of Reconstructed Image', fontsize=14)
        ax1.set_xlabel('x (λ)')
        ax1.set_ylabel('y (λ)')
        plt.colorbar(im1, ax=ax1, label='Magnitude')

        for (x_s, y_s) in sources:
            ax1.plot(x_s, y_s, 'c+', markersize=12, mew=2, label='Source Location')
        if len(sources) > 0:
            ax1.legend(loc='upper right')

        ax2 = axes[1]
        im2 = ax2.imshow(np.log10(np.abs(fft_spectrum)**2 + 1e-9),
                         cmap='coolwarm', origin='lower')
        ax2.set_title(f'{title_prefix}\n512×512 FFT Spectrum (log power)', fontsize=14)
```

```
        ax2.set_xlabel('Spatial Frequency kx')
        ax2.set_ylabel('Spatial Frequency ky')
        plt.colorbar(im2, ax=ax2, label='log10(Power)')

        plt.tight_layout()
        plt.savefig(self.fig_dir / filename, dpi=150, bbox_inches='tight')
        plt.close()

        print(f"  Saved: {filename}")

def run_complete_analysis():
    """Run complete analysis"""
    imaging_system = BackwardPropagationImaging()

    timing_results = {}

    print("\n" + "="*70)
    print("PART A: Single Point Source at (0, 0)")
    print("="*70)

    source_single = [(0, 0)]
    s_A = imaging_system.simulate_received_signal(source_single)

    print("\nMethod 1: Direct Superposition")
    t0 = perf_counter()
    img_A_direct = imaging_system.reconstruct_direct_superposition(s_A)
    t1 = perf_counter()
    timing_results['A_direct'] = t1 - t0
    print(f"  Time: {timing_results['A_direct']:.3f}s")

    fft_A_direct = imaging_system.compute_fft_spectrum(img_A_direct)
    imaging_system.plot_results(img_A_direct, fft_A_direct, source_single,
                                'Part A: Single Source - Direct Superposition',
                                'partA_single_direct.png')

    print("\nMethod 2: Convolution Method (Slow)")
    t2 = perf_counter()
    img_A_conv_slow = imaging_system.reconstruct_convolution_slow(s_A)
    t3 = perf_counter()
    timing_results['A_conv_slow'] = t3 - t2
    print(f"  Time: {timing_results['A_conv_slow']:.3f}s")

    fft_A_conv_slow = imaging_system.compute_fft_spectrum(img_A_conv_slow)
    imaging_system.plot_results(img_A_conv_slow, fft_A_conv_slow, source_single,
```

```
                            'Part A: Single Source - Convolution (Slow)',
                            'partA_single_conv_slow.png')

    print("\nMethod 3: FFT Convolution Method (Fast)")
    t4 = perf_counter()
    img_A_conv_fast = imaging_system.reconstruct_convolution_fast(s_A)
    t5 = perf_counter()
    timing_results['A_conv_fast'] = t5 - t4
    print(f"  Time: {timing_results['A_conv_fast']:.3f}s")

    fft_A_conv_fast = imaging_system.compute_fft_spectrum(img_A_conv_fast)
    imaging_system.plot_results(img_A_conv_fast, fft_A_conv_fast, source_single,
                            'Part A: Single Source - FFT Convolution (Fast)',
                            'partA_single_conv_fast.png')

    print("\n" + "="*70)
    print("PART B: Multiple Point Sources")
    print("="*70)

    sources_multiple = [
        (0.0, 15 * imaging_system.lambda_val),
        (-12 * imaging_system.lambda_val, -9 * imaging_system.lambda_val),
        (0.0, -9 * imaging_system.lambda_val),
        (12 * imaging_system.lambda_val, -9 * imaging_system.lambda_val),
    ]

    print("Source locations:")
    for i, (x, y) in enumerate(sources_multiple, 1):
        print(f"  Source {i}: ({x/imaging_system.lambda_val:.0f}λ,
{y/imaging_system.lambda_val:.0f}λ)")

    s_B = imaging_system.simulate_received_signal(sources_multiple)

    print("\nMethod 1: Direct Superposition (Multiple Sources)")
    t6 = perf_counter()
    img_B_direct = imaging_system.reconstruct_direct_superposition(s_B)
    t7 = perf_counter()
    timing_results['B_direct'] = t7 - t6
    print(f"  Time: {timing_results['B_direct']:.3f}s")

    fft_B_direct = imaging_system.compute_fft_spectrum(img_B_direct)
    imaging_system.plot_results(img_B_direct, fft_B_direct, sources_multiple,
                            'Part B: Multiple Sources - Direct Superposition',
                            'partB_multiple_direct.png')
```

```
print("\nMethod 2: Convolution Method (Slow, Multiple Sources)")
t8 = perf_counter()
img_B_conv_slow = imaging_system.reconstruct_convolution_slow(s_B)
t9 = perf_counter()
timing_results['B_conv_slow'] = t9 - t8
print(f"  Time: {timing_results['B_conv_slow']:.3f}s")

fft_B_conv_slow = imaging_system.compute_fft_spectrum(img_B_conv_slow)
imaging_system.plot_results(img_B_conv_slow, fft_B_conv_slow, sources_multiple,
                            'Part B: Multiple Sources - Convolution (Slow)',
                            'partB_multiple_conv_slow.png')

print("\nMethod 3: FFT Convolution Method (Fast, Multiple Sources)")
t10 = perf_counter()
img_B_conv_fast = imaging_system.reconstruct_convolution_fast(s_B)
t11 = perf_counter()
timing_results['B_conv_fast'] = t11 - t10
print(f"  Time: {timing_results['B_conv_fast']:.3f}s")

fft_B_conv_fast = imaging_system.compute_fft_spectrum(img_B_conv_fast)
imaging_system.plot_results(img_B_conv_fast, fft_B_conv_fast, sources_multiple,
                            'Part B: Multiple Sources - FFT Convolution (Fast)',
                            'partB_multiple_conv_fast.png')

print("\n" + "="*70)
print("PERFORMANCE SUMMARY")
print("="*70)

print(f"Part A (Single Source):")
print(f"  Direct Superposition: {timing_results['A_direct']:.3f}s")
print(f"  Convolution (Slow): {timing_results['A_conv_slow']:.3f}s")
print(f"  FFT Convolution (Fast): {timing_results['A_conv_fast']:.3f}s")

print(f"\nPart B (Multiple Sources):")
print(f"  Direct Superposition: {timing_results['B_direct']:.3f}s")
print(f"  Convolution (Slow): {timing_results['B_conv_slow']:.3f}s")
print(f"  FFT Convolution (Fast): {timing_results['B_conv_fast']:.3f}s")

if timing_results['A_conv_slow'] > 0:
    speedup_A_slow_vs_direct = timing_results['A_direct'] / timing_results['A_conv_slow']
    speedup_A_fast_vs_slow = timing_results['A_conv_slow'] / timing_results['A_conv_fast']
    speedup_A_fast_vs_direct = timing_results['A_direct'] / timing_results['A_conv_fast']
```

```
        print(f"\nPart A Speedup Ratios:")
        print(f"  Convolution/Direct: {speedup_A_slow_vs_direct:.1f}×")
        print(f"  FFT/Convolution: {speedup_A_fast_vs_slow:.1f}×")
        print(f"  FFT/Direct: {speedup_A_fast_vs_direct:.1f}×")

    if timing_results['B_conv_slow'] > 0:
        speedup_B_slow_vs_direct = timing_results['B_direct'] / timing_results['B_conv_slow']
        speedup_B_fast_vs_slow = timing_results['B_conv_slow'] / timing_results['B_conv_fast']
        speedup_B_fast_vs_direct = timing_results['B_direct'] / timing_results['B_conv_fast']

        print(f"\nPart B Speedup Ratios:")
        print(f"  Convolution/Direct: {speedup_B_slow_vs_direct:.1f}×")
        print(f"  FFT/Convolution: {speedup_B_fast_vs_slow:.1f}×")
        print(f"  FFT/Direct: {speedup_B_fast_vs_direct:.1f}×")

    print(f"\nAll images saved to '{imaging_system.fig_dir}' directory")

    return timing_results

if __name__ == "__main__":
    timing_results = run_complete_analysis()
```